

SD TCL - The micro Screen Editor

The capable one — SD BASIC highlighting, a command bar, split windows and a help screen.

```
micro {dict} file record
```

`micro` opens a record in **micro**, a full-screen editor that does rather more than edit does. Two things make it the one to use for programming:

it highlights SD BASIC	statements, reserved words, intrinsics, strings and comments, from rules SD generates out of the compiler's own tables and ships
-------------------------------	--

it has a command bar and a help screen	so everything it can do is reachable and readable without leaving the editor
---	--

It also has multiple files open at once, split windows, a plugin system and rebindable keys. **If you only want to change one line in a record**, `edit` is less to think about.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The keys below are micro's own, read from the default bindings and the help text inside the executable SD installs — **micro 2.0.15**. The SD half of the page describes SD Core for Windows W1.0-0.

Both editors are installed with SD

You do not install anything. SD's installer checks for both editors and installs whichever is missing, machine-wide, so every account SD creates can reach them.

If that could not happen — an offline machine, or one whose policy blocks the package manager — the verb says so and names the command that installs it, rather than opening nothing and reporting the record unchanged.

The keys

Ctrl-S saves, **Ctrl-Q** quits, and **Ctrl-G** **opens the help screen**, which is the answer to everything not on this page.

Ctrl-S	save
Ctrl-Q	close the file — and quit, if it is the last one open
Ctrl-G	open and close help
Ctrl-E	the command bar — see below
Alt-G	show and hide the key hints
Ctrl-Z · Ctrl-Y	undo · redo
Ctrl-X · Ctrl-C · Ctrl-V	cut · copy · paste
Ctrl-K · Ctrl-D	cut the current line · duplicate it
Ctrl-A	select all
Ctrl-F	find
Ctrl-N · Ctrl-P	find next · find previous
Ctrl-O	open a file
Ctrl-T	open a new tab
Alt-A · Alt-E	start · end of line
Alt-B · Alt-F	previous · next word

Function keys work too, for anyone who prefers them: **F1** help, **F2** save, **F3** or **F7** find, **F4** or **F10** quit.

Ctrl-Q **closes the file rather than the editor**. With one record open — which is how SD starts it — that is the same thing. With a second file opened by hand it is not, and micro stays up until the last one is closed.

The command bar

`Ctrl-E` opens a `>` prompt at the foot of the screen. It takes micro's own commands, and the two worth knowing on the first day are:

<code>> help</code>	the same screen as <code>Ctrl-G</code>
<code>> help keybindings</code>	every key, including the ones not listed above
<code>> set option value</code>	change a setting for this session

Saving is broken for an ordinary account in W1.0-0

`micro` **draws, edits and highlights correctly and then cannot save**, unless the session is elevated:

```
Permission denied. Save with sudo not supported on Windows
```

The record you were editing is not touched and nothing is lost — the failure is on micro's side of the working copy, and quitting without saving leaves the record exactly as it was.

It is not a permission problem with your file or your account. SD points micro's configuration directory at a folder under `C:\Program Files`, which micro has to write to and an ordinary account may not. It is recorded as a release blocker in the project's fix lists.

Until it is fixed, use `ed` for anything you intend to save. It runs inside SD, needs no external editor and no operating-system access, and is documented on SD TCL - The ed Line Editor. `micro` is still useful for reading a record with syntax highlighting.

Highlighting SD BASIC

SD SHIPS THE RULES AND POINTS micro AT THEM. They are generated from the BASIC compiler's own tables rather than typed by hand, so the list of statements, reserved words and intrinsics is the one the compiler actually accepts.

It works on a record from a `bp` file and not on other records, and the reason is worth knowing: micro decides which language a file is by matching its *name*, and a record called `MY.REPORT` tells it nothing. So SD names the working copy `*record*.editing.sdbasic`, and that suffix is what micro recognises. A record edited out of any other file is plain text.

What SD does around the editor

The two screen editors are **one SD program with two names**. The working copy, the save and compile questions, the two permission gates and the refusals are the same for both, and they are set out on SD TCL - The edit Screen Editor.

In short: the record is copied into `$hold`, `micro` is run on the copy, and on the way out SD asks whether to save it and — for a `bp` record — whether to compile and catalogue it. Both verbs need `OS.EXECUTE` permission and a real terminal, and refuse a session driven down a pipe:

```
:micro bp zzed
micro needs a terminal to draw on, and this session has none.
ed, the line editor, works anywhere.
```

Marks, and how to type one

A **field** mark is a line break, so `micro` handles fields on its own. The other three have a token you type instead. **Every token is `~` and one more character, and `~` is the only escape character**.

<code>~~</code>	a value mark
<code>~`</code>	a subvalue mark
<code>~!</code>	a text mark
<code>~-</code>	a literal <code>~</code> , where one would otherwise be misread
<code>~,</code>	a literal <code>,</code> , where one would otherwise read as a separator

Marks in a row are separated by a comma — a text mark, a text mark and a value mark is `~!,~!,~~`. The conversion is lossless: no record is refused and none is mangled. The full rules are on SD TCL - The edit Screen Editor.

Who has these verbs

`micro`, `edit` and `ed` are all **programmer verbs**. A standard account has none of them.

See also

[SD TCL - The edit Screen Editor](#) · [SD TCL - The ed Line Editor](#) · [SD TCL - Programs and the Catalogue](#).